

04_medallion

August 18, 2026

1 NB4 — Medallion Pipeline (Bronze → Silver → Gold), lightweight

Use case: LLM observability — exact schema from slide §8 (Lakehouse cho AI/ML) medallion frame. Maps to deliverable bullet 4 (the Milestone-1 Lakehouse artifact).

Pre-req: ran `make data` — but if you jumped straight here, the cell below generates the Bronze sample for you rather than failing on a missing path.

```
[1]: import _setup # noqa: F401 -- adds scripts/ to sys.path
from pathlib import Path

import polars as pl
import duckdb
from deltalake import DeltaTable, write_deltalake
from lakehouse import path, reset

BRONZE = path("bronze", "llm_calls_raw")
SILVER = path("silver", "llm_calls")
GOLD   = path("gold", "llm_daily_metrics")

# Self-healing pre-req (same pattern as NB7/NB8). Without this, skipping
# `make data` surfaces as a raw `Ds {code: 2, kind: NotFound}` from the Rust
# layer - technically correct, useless to a student.
if not Path(BRONZE).exists():
    print("Bronze not found - running scripts/generate_data_lite.py first ...")
    import generate_data_lite

    generate_data_lite.main()
```

1.1 Bronze — verify raw is loaded

```
[2]: bronze_n = DeltaTable(BRONZE).to_pyarrow_table().num_rows
print(f"Bronze rows: {bronze_n:,}")
print(pl.from_arrow(DeltaTable(BRONZE).to_pyarrow_table().slice(0, 2)))
```

Bronze rows: 200,000
shape: (2, 3)

request_id	ts	raw_json
---	---	---
str	datetime[s, UTC]	str
63d3c82e-b64a-4bb2-93c8-8bcf87...	2026-04-01 00:00:00 UTC	{"model": "claude-sonnet-4-6", ...
ddd78a95-eed0-4b67-9d92-de668a...	2026-04-01 00:00:03 UTC	{"model": "claude-haiku-4-5", ...

1.2 Silver — parse, validate, dedup

Rules: drop malformed JSON, dedupe by `request_id`, project typed columns.

```
[3]: reset(SILVER)

# DuckDB does the JSON parse + dedup in one query - Polars also works,
# DuckDB just has nicer JSON syntax for this case.
# DuckDB reads Delta through Arrow, not through `delta_scan()`. The latter
# autoloads an extension over the network; Arrow registration is offline and
# zero-copy, so the lab works on a locked-down machine.
con = duckdb.connect()
con.register("bronze", DeltaTable(BRONZE).to_pyarrow_table())

silver_arrow = con.sql(f"""
    WITH parsed AS (
        SELECT
            request_id,
            ts,
            CAST(ts AS DATE) AS date,
            json_extract_string(raw_json, '$.model') AS model,
            json_extract_string(raw_json, '$.user_id') AS user_id,
            CAST(json_extract(raw_json, '$.usage.input') AS INTEGER) AS
↳prompt_tokens,
            CAST(json_extract(raw_json, '$.usage.output') AS INTEGER) AS
↳completion_tokens,
            CAST(json_extract(raw_json, '$.latency_ms') AS INTEGER) AS latency_ms,
            json_extract_string(raw_json, '$.status') AS status,
            ROW_NUMBER() OVER (PARTITION BY request_id ORDER BY ts) AS rn
        FROM bronze
    )
    SELECT request_id, ts, date, model, user_id,
```

```

        prompt_tokens, completion_tokens, latency_ms, status
    FROM parsed
    WHERE rn = 1 AND model IS NOT NULL
    """).arrow()

write_deltalake(SILVER, silver_arrow, mode="overwrite", partition_by=["date"])

silver_n = DeltaTable(SILVER).to_pyarrow_table().num_rows
print(f"Silver rows: {silver_n:,} (Bronze {bronze_n:,} → dedup dropped_{bronze_n - silver_n:,})")
assert silver_n < bronze_n, (
    "Silver has the same row count as Bronze - dedup did not run. "
    "Did you regenerate Bronze with the latest generator (which injects_{retries})?"
)

```

Silver rows: 190,052 (Bronze 200,000 → dedup dropped 9,948)

1.3 Gold — aggregate to (date, model) metrics

```

[4]: reset(GOLD)

# Illustrative cost model - NOT canonical pricing.
# (input USD / 1M tokens, output USD / 1M tokens)
COST_TABLE = """
    VALUES
        ('claude-haiku-4-5', 0.80, 4.00),
        ('claude-sonnet-4-6', 3.00, 15.00),
        ('claude-opus-4-7', 15.00, 75.00)
    """

con.register("silver", DeltaTable(SILVER).to_pyarrow_table())
gold_arrow = con.sql(f"""
    WITH cost(model, c_in, c_out) AS ({COST_TABLE})
    SELECT
        s.date,
        s.model,
        QUANTILE_CONT(s.latency_ms, 0.50) AS p50_latency_ms,
        QUANTILE_CONT(s.latency_ms, 0.95) AS p95_latency_ms,
        SUM(s.prompt_tokens) AS total_prompt_tokens,
        SUM(s.completion_tokens) AS total_completion_tokens,
        AVG(CASE WHEN s.status <> 'ok' THEN 1.0 ELSE 0.0 END) AS error_rate,
        (SUM(s.prompt_tokens) * c.c_in / 1e6) +
        (SUM(s.completion_tokens) * c.c_out / 1e6) AS cost_usd
    FROM silver s
    JOIN cost c USING (model)
    GROUP BY s.date, s.model, c.c_in, c.c_out
    """)

```

```

        ORDER BY s.date, s.model
    """).arrow()

write_deltalake(GOLD, gold_arrow, mode="overwrite", partition_by=["date"])

# Z-order for fast filter-by-model dashboards
DeltaTable(GOLD).optimize.z_order(["model"])

```

```

[4]: {'numFilesAdded': 8,
      'numFilesRemoved': 8,
      'filesAdded':
      {'avg':2768.75,"max":2770,"min":2766,"totalFiles":8,"totalSize":22150}',
      'filesRemoved':
      {'avg':2687.875,"max":2689,"min":2687,"totalFiles":8,"totalSize":21503}',
      'partitionsOptimized': 8,
      'numBatches': 8,
      'totalConsideredFiles': 8,
      'totalFilesSkipped': 0,
      'preserveInsertionOrder': False,
      'plannerStrategy': 'zOrder',
      'preservedStableOrder': False,
      'maxBinSpanFiles': 1}

```

1.4 Verify Gold

```

[5]: gold_df = pl.from_arrow(DeltaTable(GOLD).to_pyarrow_table())
      print(gold_df)

# Slide-5 deliverable: "Gold p50/p95/cost qua 7 ngày". Make that explicit.
n_dates = gold_df.select("date").n_unique()
n_models = gold_df.select("model").n_unique()
print(
    f"\n Gold deliverable metrics \n"
    f" Distinct dates: {n_dates:>3} (target 7)\n"
    f" Distinct models: {n_models:>3}\n"
    f" Total Gold rows: {gold_df.height:>3} (= dates × models)"
)
assert n_dates >= 7, (
    f"Gold has only {n_dates} dates - slide deliverable requires 7. "
    "Re-run `make data` (the generator spreads across 7 UTC days)."
)

```

shape: (24, 8)

date	model	p50_latenc	p95_laten	total_pro	total_com
error_rat	cost_usd				
---	---	y_ms	cy_ms	mpt_token	pletion_t e

date f64	str	---	---	s	okens	---
		f64	f64	---	---	f64
				decimal[3 8,0]	decimal[3 8,0]	
2026-04-04 0.049224	claude-hai 44.62453 ku-4-5	559.0	1125.9	15997428	7956647	
2026-04-04 0.052158	claude-opu 291.85569 s-4-7	2995.5	5940.05	5586541	2774101	
2026-04-04 0.047897	claude-son 348.73727 net-4-6	1383.0	2747.0	33198197	16609512	
1 2026-04-02 0.049327	claude-hai 46.305166 ku-4-5	565.0	1140.0	16651653	8245961	
2026-04-02 0.052803	claude-opu 291.28575 s-4-7	3004.0	5972.0	5579305	2767949	
...	...	...	...	...	...	...
2026-04-03 0.043369	claude-opu 297.05086 s-4-7	3037.0	5976.65	5601091	2840460	
5 2026-04-03 0.046578	claude-son 347.65212 net-4-6	1388.0	2732.0	33186257	16539557	
6 2026-04-08 0.042171	claude-hai 13.496638 ku-4-5	574.0	1118.3	4855783	2403003	
2026-04-08 0.059367	claude-opu 80.587485 s-4-7	2945.5	6058.05	1571994	760101	

2026-04-08	claude-son	1404.0	2782.85	9596279	4882116
0.049979	102.02057				
	net-4-6				

7

Gold deliverable metrics

Distinct dates:	8	(target 7)
Distinct models:	3	
Total Gold rows:	24	(= dates × models)

1.5 Deliverable check

- ☐ All three tables exist under `_lakehouse/{bronze,silver,gold}/`
- ☐ Silver has fewer rows than Bronze (dedup worked)
- ☐ Gold spans 7 dates × 3 models (slide §8 medallion contract)
- ☐ Cost & error_rate columns populated and non-zero